

ADI Internship

AMBA 3 AHB-Lite Architecture and Design Specification

Name: Yossef Medhat Ali

Submitted to: Eng. Fatma Ali

July 23, 2026

Introduction

This document outlines the complete system protocol features, hardware architecture, and Finite State Machine (FSM) designs for a full AMBA 3 AHB-Lite system. It covers the Master, Slave, and Interconnect (Decoder and Multiplexor) modules as defined by the AMBA 3 AHB-Lite Protocol v1.0 specification.

Features of AMBA 3 AHB-Lite

The AMBA AHB-Lite protocol is an interface built for high-performance synthesizable designs.

Supported Protocol Features

- **Single Master Operation:** The protocol fundamentally supports a single bus master.
- **High-Bandwidth Operation:** Designed for high clock frequency systems.
- **Burst Transfers:** Supports fixed-length bursts (4, 8, and 16 beats) that can be incrementing or wrapping, alongside undefined length incrementing bursts.
- **Single-clock Edge Operation:** All signal timings trigger on the rising edge of the bus clock (HCLK).
- **Non-tristate Implementation:** Uses a central multiplexor scheme with separate read and write buses, avoiding tristate drivers.
- **Data Bus Configurations:** The protocol allows the AHB-Lite data bus to be 8, 16, 32, 64, 128, 256, 512, or 1024-bits wide. However, it is recommended that you use a minimum bus width of 32 bits. A maximum bus width of 256 bits is adequate for almost all applications.
- **Pipelined Operations:** The address phase of a transfer occurs simultaneously with the data phase of the previous transfer.
- **Protection Control:** HPROT signals distinguish between privileged/user accesses, opcode/data fetches, and cacheable/bufferable accesses.
- **Wait States:** Slaves can delay data phase completion by driving HREADY LOW.
- **Locked Transfers:** Facilitates indivisible transfer sequences via the HMASTLOCK signal.

Hardware Architecture Description

An AHB-Lite system is built using a Master, one or more Slaves, and the Bus Interconnection (Address Decoder and Read Data).

(We can assume a microcontroller is writing/reading a memory)

Master Architecture

The Master receives a write/read request from the microcontroller:

Signal Name	Width	Description
Request Interface		
i_req_valid	1-bit	Asserts HIGH when the internal logic has a valid transfer request.
i_req_write	1-bit	Determines transfer direction: 1 for Write, 0 for Read.
i_req_addr	32-bit	The target system address for the transfer.
i_req_wdata	32-bit	The data payload to be written (ignored during reads).
i_req_size	3-bit	Dictates the transfer size to map directly to HSIZE (e.g., byte, halfword, word).
Feedback & Response Interface		
o_req_ready	1-bit	Flow control. Asserts HIGH when the Master is in IDLE and ready to accept a new request. If LOW, the internal logic must hold i_req_valid and the address/data stable.
o_resp_valid	1-bit	Asserts HIGH for one clock cycle when the AHB transfer is fully completed (tied to HREADY going HIGH in the Data Phase).
o_resp_rdata	32-bit	The valid read data fetched from the AHB bus (maps to HRDATA).
o_resp_error	1-bit	Asserts HIGH if the AHB transfer failed (maps directly to HRESP).

Table 1: Master Internal Frontend Interface Signals

Then it initiates a transfer by driving address and control signals.

- **Control & Address:** Generates HADDR, HWRITE, HSIZE, HBURST, HTRANS, HPROT, and HMASTLOCK.
- **Data:** Drives write data on HWDATA and samples read data from HRDATA.
- **Response:** Samples HREADY to determine transfer extension and HRESP for status (OKAY or ERROR).

Slave Architecture

The Slave responds to master-initiated transfers.

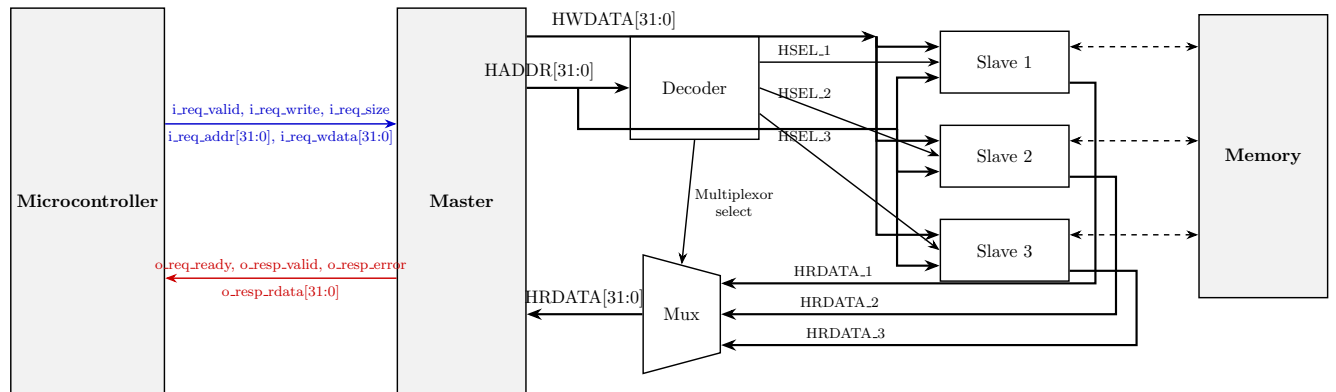
- **Selection:** Activated by the HSELx signal generated by the decoder.
- **Data:** Samples HWDATA during writes and drives HRDATA during reads.
- **Response:** Uses HREADYOUT to insert wait states and HRESP to signal success or failure.

Interconnect Architecture

The interconnect routes data and control between the Master and multiple Slaves.

- **Address Decoder:** A centralized decoder that monitors HADDR to generate the appropriate slave select signal (HSELx). It also supplies multiplexor control signals.
- **Read Data Multiplexor:** A centralized unit that routes the active slave's HRDATA, HREADYOUT, and HRESP back to the master's HRDATA, HREADY, and HRESP inputs.

Block Diagram



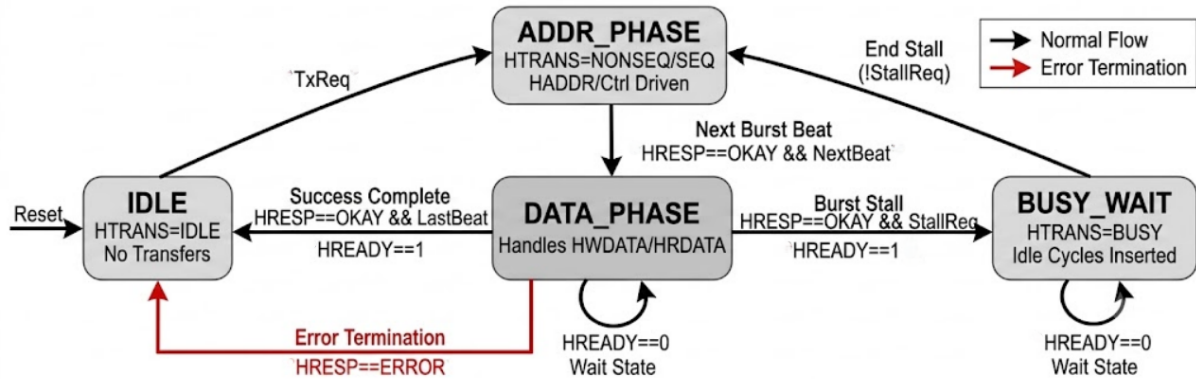
System FSM Design Description

Master FSM:

The Master FSM handles the two-phase pipeline architecture.

1. **IDLE:** HTRANS is set to IDLE; no data transfers are requested.
2. **ADDR_PHASE:** Drives HADDR and control signals (HWRITE, HSIZE, HBURST). HTRANS is set to NONSEQ for singles/first burst beats, or SEQ for subsequent burst beats. Always lasts one HCLK cycle.
3. **DATA_PHASE:** Overlaps with the next address phase. Drives HWDATA (writes) or samples HRDATA (reads). Monitors HREADY:
 - $HREADY == 0$: Wait state. Master must hold write data and keep next address/control stable.
 - $HREADY == 1 \ \&\& \ HRESP == OKAY$: Transfer successful. Move to the next burst beat or IDLE.
 - $HREADY == 1 \ \&\& \ HRESP == ERROR$: Transfer failed. Master can optionally cancel remaining burst beats.
4. **BUSY_WAIT:** Drives HTRANS = BUSY to insert idle cycles during a burst.

Master-Side AHB-Lite FSM State Machine Diagram



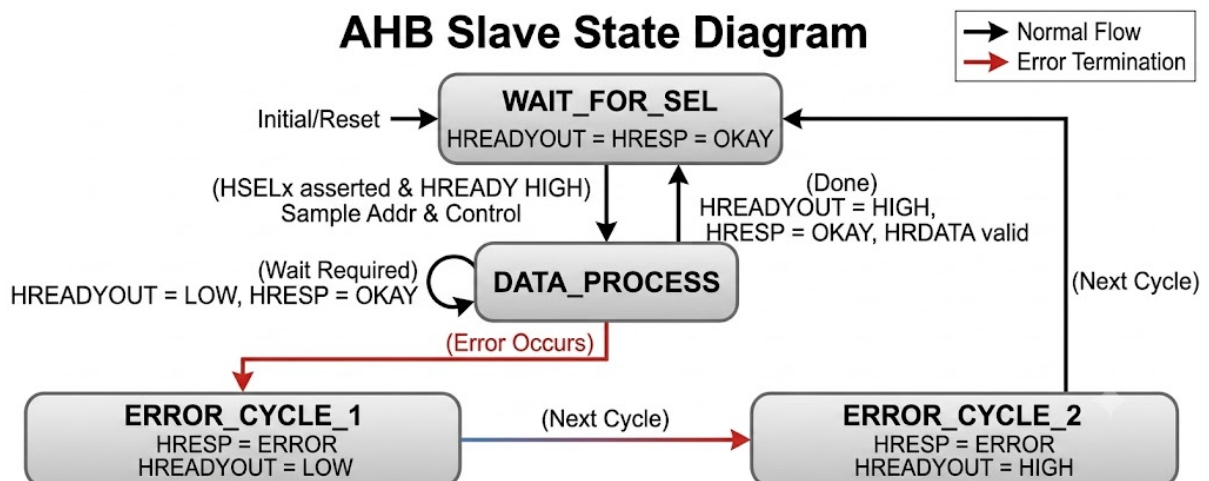
The TxReq and StallReq comes from the microcontroller

Slave FSM:

The Slave FSM processes the incoming pipelined requests.

1. **WAIT_FOR_SEL:** Monitors HSELx and HREADY. The slave must only sample address and control signals when HSELx is asserted and HREADY is HIGH.
2. **DATA_PROCESS:** Once selected, the slave evaluates the requested operation.
 - If more time is needed, the slave drives HREADYOUT = LOW and HRESP = OKAY to insert wait states.
 - Upon completion, drives HREADYOUT = HIGH and HRESP = OKAY. Valid data is placed on HRDATA for reads.
3. **ERROR_CYCLE_1:** If an error occurs, the slave must execute a two-cycle error response. In cycle 1, the slave drives HRESP = ERROR and HREADYOUT = LOW.
4. **ERROR_CYCLE_2:** In the second cycle, the slave drives HRESP = ERROR and HREADYOUT = HIGH to end the transfer.

AHB Slave State Diagram



System Verification and Test Cases:

Test Case 1: Address Decoding and Data Routing

Objective: Verify that the interconnect's address decoder correctly routes requests to the appropriate slave based on the memory map, and that the multiplexor properly returns the correct **HRDATA**.

Procedure:

- Issue sequential **SINGLE** write transfers to the base addresses of Slave 0 (e.g., 0x0000_0000), Slave 1 (e.g., 0x0000_0400), and Slave 2 (e.g., 0x0000_0800).
- Perform corresponding read transfers from the exact same addresses.
- **Expected Outcome:** The interconnect asserts the correct **HSELx** signal for each address range. The read data matches the written data without any cross-slave memory corruption.

Test Case 2: Sequential Burst Transfers (INCR4)

Objective: Validate the pipelined overlapping of the Address and Data phases during a continuous burst transfer.

Procedure:

- Initiate an **INCR4** burst write operation, driving a new data word (**HWDATA**) on every active clock cycle without wait states.
- Read back the four addresses sequentially to verify the internal address auto-increment logic.
- **Expected Outcome:** The Master successfully increments the address. The target Slave receives and stores all four beats accurately.

Test Case 3: Narrow Transfers (Byte and Halfword)

Objective: Verify the byte-enable masking logic within the Slave memory to ensure partial-word writes do not destructively overwrite adjacent data in the 32-bit array.

Procedure:

- Initialize a 32-bit memory word to all zeros.
- Execute an 8-bit (Byte) write to the lowest byte offset (**HSIZE** = 3'b000).
- Execute a 16-bit (Halfword) write to the upper two byte offsets (**HSIZE** = 3'b001).
- Read the full 32-bit word back from the bus.
- **Expected Outcome:** The unwritten byte lane retains its initialized zero value, while the written lanes reflect the newly written data, proving proper byte-lane isolation and decoding.

Test Case 4: Master BUSY Wait State Insertion

Objective: Confirm the Master FSM's ability to stall a burst transfer mid-sequence and maintain bus stability when the driving microcontroller requires a pause.

Procedure:

- Start an **INCR4** burst transfer.
- Immediately following the completion of the first beat (when **HREADY** goes HIGH), assert the microcontroller's internal busy signal.
- Hold the busy state for multiple clock cycles before clearing it to resume the burst.
- **Expected Outcome:** The Master transitions smoothly to the **BUSY_WAIT** state, driving **HTRANS** = **BUSY** to the bus. The Slave is held stable without capturing invalid data, and the burst resumes seamlessly to completion once the busy signal clears.